

CAI Negotiation Assignment - Group 33 Final Report

Maja Czerwińska, Paulina Sobocińska, Eli Shamayev, and Lukáš Chládek

Abstract

This report describes a novel implementation of an autonomous agent for bilateral negotiation using a BOA (bidding strategy, opponent model, acceptance strategy) architecture. The authors demonstrate that an agent with an opponent-strategy model and the appropriate decision-making strategy can outperform a wide range of ANAC 2023 agents.

1 Introduction

Multi-agent negotiation is a task for artificially intelligent and autonomous systems that involves consensus decision-making. This report focuses on bilateral negotiations between two agents supporting SAOP, the Stacked Alternating Offers Protocol (Aydogan et al. 2017).

Our task, creating an agent for the Automated Negotiating Agents Competition (ANAC) 2023, requires navigating 50 domains with additive linear issue spaces. Bids are placed in alternating order with one opponent. No communication besides bidding and acceptance of offers may take place—therefore, agents can only find out about their opponent’s goals and strategy through progress in the negotiation.

2 Design

Our agent’s goal is optimization of its individual utility. To do this, it must make competitive bids and concede minimally, finding the highest-utility bid that the opponent will also accept. A key part of this is identifying the opponent’s strategy.

To accomplish this, our agent is designed around a BOA architecture (Baarslag 2016). Its architecture is composed of the bidding strategy, opponent model, and acceptance strategy as three high-level modules. BOA was chosen because it excels in flexibility, allowing us to create a more effective agent by combining innovations to each of its parts. This section will describe them in sequence.

2.1 Base implementation

The agent was implemented in Python 3.9 and is fully compatible with the GeniusWeb capabilities in use for the Automated Negotiation League (ANL) 2023. All contributions by the authors of this report are extensions of the `TemplateAgent`

package in the ANL 2023 example agent repository¹ by Bram Renting and collaborators. This section describes the structure of our implementation and corresponding methods in the source code.

2.2 Baseline opponent model

If the negotiation domain Ω is said to contain all possible offers $\omega \in \Omega$, the agent already knows its own utility function $u_m : \Omega \rightarrow \mathbb{R}$ and thus also the utility $u_m(\omega)$ for all such offers. However, the opponent’s function $u_o : \Omega \rightarrow \mathbb{R}$ and values $u_o(\omega)$ are unknown. As defined by Baarslag (2016), the principle of the opponent model is to predict $u_o(\omega)$.

Our agent utilizes the development template for ANAC 2023, which implements a statistical opponent model. The `IssueEstimator` and `ValueEstimator` classes are part of the `TemplateAgent`, and their functionality will not be discussed here in detail. The implementation uses frequency of received bids to provide an estimator $\hat{u}_o : \Omega \rightarrow \mathbb{R}$ for the opponent’s utility. The implementation supports linear additive utility spaces only.

2.3 DANS extended opponent model

As an extension of the provided estimator $\hat{u}_o$, we use DANS analysis to identify the opponent’s behavior based on the bid history. DANS is a data-driven approach that uses the opponent’s bid change between consecutive time steps to classify bidding behavior. We use a method adapted from Li, Murukanah, and Jonker (2021).

The process assigns a category verdict to each bid based on the current and previous time step. In doing this, DANS considers only the opponent’s offers. First, take the last two offers in the opponent’s bid history B_o :

$$\omega_i, \omega_{i-1} \text{ where } B_o = \{\dots, \omega_{i-1}, \omega_i\}$$

Next, define the two values to be used in classifying the behavior:

- Δm for change in our agent’s utility:

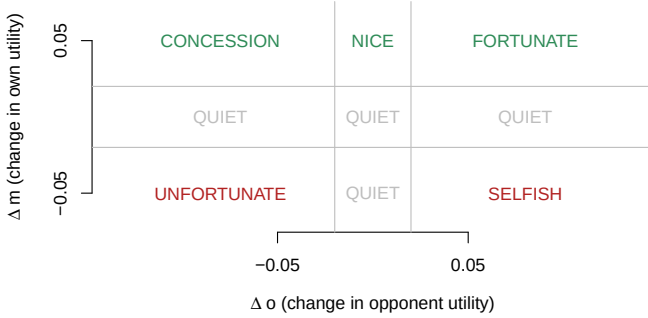
$$\Delta m = u_s(\omega_i) - u_s(\omega_{i-1})$$

- Δo for change in the opponent agent’s utility (using the estimator):

¹<https://github.com/brenting/ANL-2023-example-agent>

$$\Delta o = \hat{u}_o(\omega_i) - \hat{u}_o(\omega_{i-1})$$

Using thresholds on these values, one class is selected from the following five: FORTUNATE, UNFORTUNATE, SELFISH, NICE, and SILENT. These describe the likely cause of the opponent’s bid. The time-series history of DANS classes is then used in the bidding and acceptance strategy to decide on the best next action, such as by judging the dominant characteristic of the opponent. We consider this data an extension of the opponent model, as it serves to inform the decision-making process.

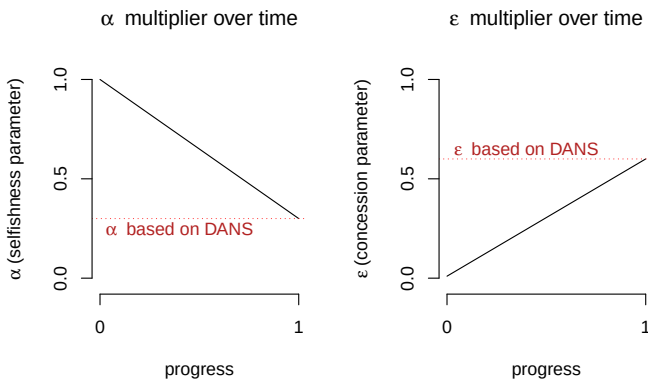


2.4 Bidding strategy

Faratin, Sierra, and Jennings (1998) propose two approaches to the question of bidding: time-based, where the agent concedes more as the deadline approaches, and behavior-based, where each bid is a response to opponent behavior.

Our bidding strategy combines the two, making use of all available information.

First, we define a function $\rho : (\Omega, \mathbb{R}, \mathbb{R}) \rightarrow \mathbb{R}$ that computes a numeric score for each bid. This score is distinct from the bid utility in that it instead represents how suitable the bid is for this stage of the negotiation. Its numeric parameters are α (the selfishness of the bid) and ϵ (the concession rate), which are given by the DANS classification and a multiplier that changes as the negotiation progresses.



To find the next bid to offer, the agent searches 1000 random bids from the domain, and selects the best one based on two conditions: it has the highest score based on the **score_bid(alpha, eps)** function, and our utility from this bid is higher than the threshold we set. If no bid satisfies the threshold, the agent falls back to proposing the best-scoring

bid overall to ensure continued progress. The threshold is computed dynamically. First we compute a time-based threshold as below:

$$b = b_{\text{early}} + (b_{\text{late}} - b_{\text{early}}) \times \text{progress}$$

$$\text{baseline-threshold} = U_{\min} + (U_{\max} - U_{\min}) \times (1.0 - \text{progress}^{(1.0/b)})$$

where b controls the concession speed over time and $U_{\max}/U_{\min}$ are the utility bounds.

Then we adapt this time-based threshold by identifying the most recent DANS category of our opponent. We look at the most common DANS category from the last 5 bids. This allows us to adapt to the opponent’s current strategy rather than reacting to their overall behavior across the session. Each DANS type has an associated offset, which we use to adjust our final threshold as follows:

$$\text{threshold} = \text{baseline-threshold} \times (1 + \text{offset})$$

α and ϵ are parameters that control our selfishness and concession rate, and they are also chosen dynamically in response to our opponent’s behavior and then linearly interpolated over time to allow more concession as the deadline approaches. We get the two most common DANS types in the last 5 bids, and again each DANS type has a predefined alpha and eps that we will use in response. We compute a weighted average of these values, and then adjust them based on time progress.

All parameters in these functions were optimized later on using Bayesian Optimization.

2.5 Acceptance strategy

Empirical evidence presented by Baarslag, Hindriks, and Jonker (2014) demonstrates that among the basic acceptance strategies, AC_{next} consistently outperforms alternatives such as AC_{const} or AC_{time} , both in terms of agreement rates and the average utility achieved. This strategy accepts an opponent’s offer if it exceeds the utility of the agent’s next planned bid, promoting responsive and adaptive negotiation behavior.

However, AC_{next} can fall short in scenarios where opponents begin to make significantly better offers toward the end of the negotiation. To address this, Baarslag et al. explore composite strategies that incorporate aggregate measures of offer quality over time. Among these, $AC_{\text{combi}}(T, \text{MAX}^T)$ — which accepts if the current offer is the best seen so far — stands out as a particularly aggressive yet promising option. It moves the basis for acceptance from immediate short-term comparison to a more historical, objective reference, effectively rewarding opponents who begin to concede later in the negotiation.

Out of the aggregate-based approaches considered in the study — including MAX^W , AVG^W , and MAX^T — MAX^T focuses solely on the top historical utility. While this makes it a strong candidate for maximizing potential gain, the authors caution that such a strict threshold can result in fewer successful agreements. Offers that fail to meet the maximal utility requirement are rejected, increasing the risk of negotiations ending without a deal.

To strike a balance between assertiveness and practicality, our agent implements a three-phase acceptance strategy that combines these insights.

Phase 1 (0%–90%): The agent uses AC_{next} , comparing the utility of the opponent’s current bid to its own next planned bid. If the opponent’s offer is better, the agent accepts. This maintains a competitive posture while allowing the agent to capitalize on unexpectedly favorable proposals.

Phase 2 (90%–98%): The acceptance condition shifts to $AC_{\text{combi}}(T, \text{MAX}^T)$. This allows the agent to recognize concessions from the opponent by checking whether the current bid is the best received so far. It reflects a shift in strategy toward the end of the negotiation — from outperforming our own plan to evaluating the opponent’s historical behavior. While Baarslag, Hindriks, and Jonker (2014) primarily apply the $AC_{\text{combi}}(T, \text{MAX}^T)$ condition in the final round of negotiation, our implementation adopts it earlier — from 90% of the timeline onward. This adjustment was made to allow the agent to recognize and accept optimal bids that may arrive just before the final round, reducing the risk of missing high-utility offers due to timing. Though this differs from the original setup, it preserves the theoretical motivation behind MAX^T — rewarding cooperation and capturing historical peak offers — while increasing responsiveness to late concessions from the opponent.

Phase 3 (after 98%): As a fallback mechanism, the agent applies AC_{const} with a threshold of 0.6, accepting any offer with utility above this value. This prevents the negotiation from ending without agreement due to overly rigid acceptance criteria, while still maintaining a baseline level of fairness.

This phased approach is informed by empirical findings and aims to maximize utility without compromising the likelihood of reaching an agreement — maintaining pressure when it’s advantageous but avoiding unnecessary failures

3 Optimization

The implementation of our agent includes tuning parameters (the full list can be found in Appendix A). To find an optimum value for each one, we decided to use Bayesian Optimization. Our objective was to maximize our average utility. We first ran 12 iterations using random uniform sampling to explore the parameter space and gather initial data for model fitting. This allowed us to approximate the shape of the function that has to be maximized. After that, we ran 22 iterations that tried to find the maximum of our function. In each iteration, we fit the GaussianProcessRegressor model on the previously obtained results, and we iteratively tried to find the best possible combinations of our parameters. Each iteration ran a negotiation session between our agent and 49 opponents that can be found in Appendix A. To avoid overfitting on one domain, we randomly sampled 2 domains for each iteration, so in total we ran 98 negotiation sessions per iteration, and 3332 in total. The change in the average utility value of our agent across all iterations can be seen in the plot below.

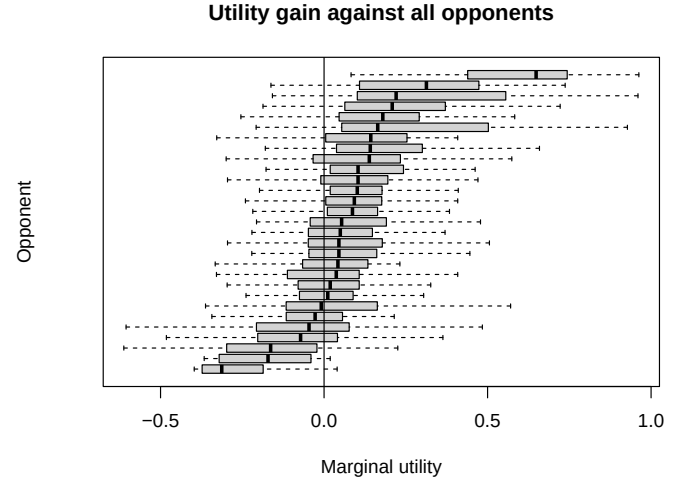
4 Evaluation

After optimizing the agent and fixing its parameters, we performed a systematic evaluation against all other agents in the negotiation domains provided. We ran a total of 5000 negotiations with a 10 second negotiation time each. The total comes from the cross product of 50 opponent agents, 50 domains,

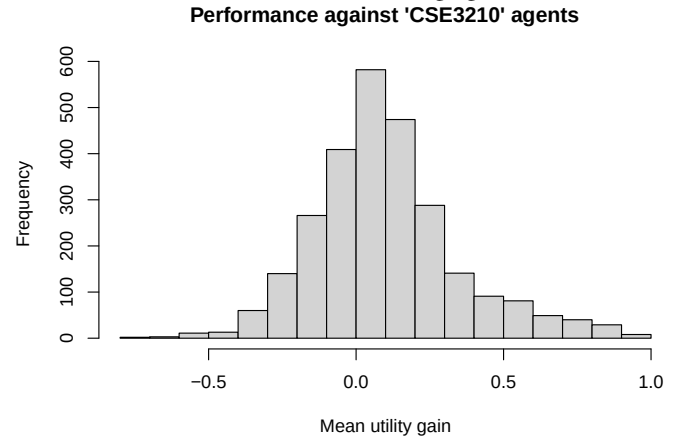
and 2 ‘orientations’ for each domain. The data that results from this testing is discussed in this chapter as an indicator of our agent’s performance and limitations.

4.1 Performance summary against other agents

The horizontal box plot below shows the marginal utility against all opponents over 5000 runs. Negotiations where the agents did not reach consensus are assumed to have utility 0 (the impact of this is noted in the evaluation section). Each line in the box plot represents one opponent, and if biased to the positive side indicates the relative strength of our agent. Fairness of the evaluation was ensured through simulating both ‘orientations’ of each domain, so as to balance out its inherent bias. Any positive or negative bias is therefore representative of the strength of the agent.



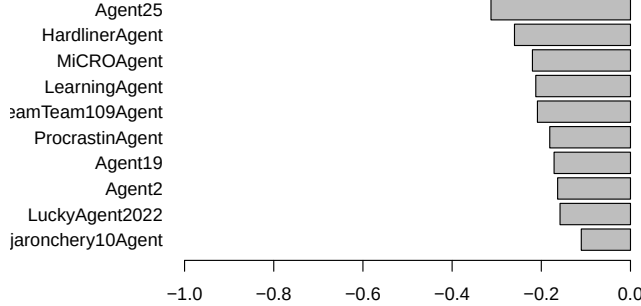
The following plot evaluates the agent against agents marked ‘CSE3210’ in the repository. The histogram shows the occurrence of runs with a certain marginal utility. The spread of this plot is given by the diverse range of agents and utility spaces. Crucially, however, the skewness indicates above-average performance in the negotiation tournament as the utility was systematically higher than that of opponents, which is desired behavior for a self-serving agent.



The following two plots compare our agent’s performance against the 10 empirically toughest and easiest opponents. This was determined by ranking them according to the median marginal utility: tough opponents have a higher utility than

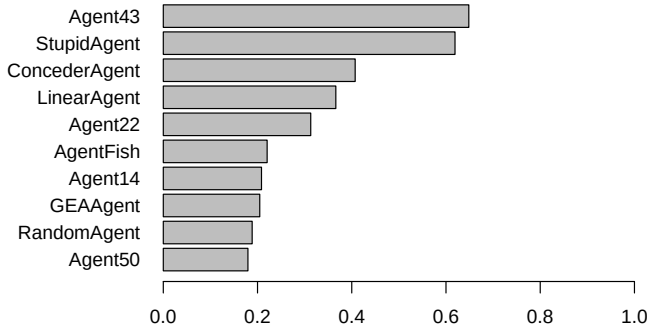
our agent in the average case, while easy ones. The agent falls short of consistently outranking some of the most advanced implementations, which is discussed in the limitations section.

Median utility gain against 10 toughest opponents



On the other hand, it is notable that our agent performs well with diverse classes of ‘simple’ agent, in particular the *conceder*, *boulware*, and *linear* agents. This indicates adaptability to several strategies. We consider this a successful application of strategy detection, as our implementation was able to use the DANS methodology to adapt to opponents.

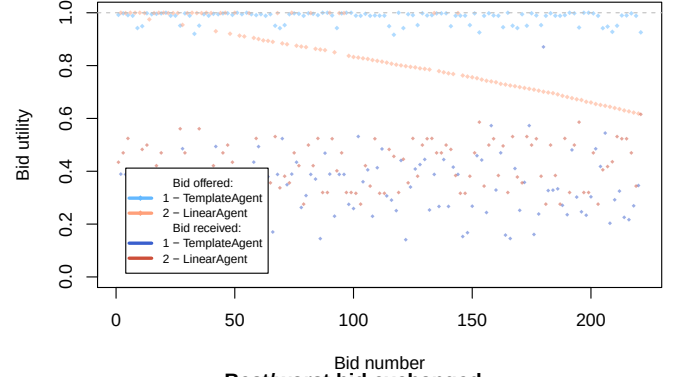
Median utility gain against 10 easiest opponents



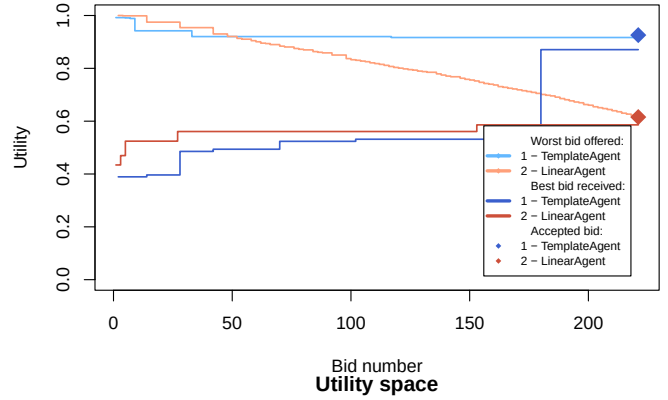
4.2 Analysis of bidding behavior

For a more in-depth analysis of the bidding behavior, we select a negotiation that occurred between our agent (marked *TemplateAgent*) and *LinearAgent*, which bids using a linear concession strategy. In this instance of the simulation, our agent beats out *LinearAgent* with 0.93 utility (ours) compared to 0.616 (theirs). This result was chosen for further analysis because its domain (*domain26*) has opposed goals (opposition: 0.339) and a relatively balanced distribution.

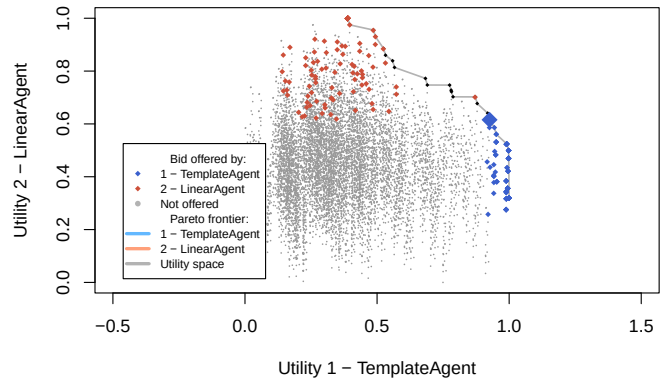
Bids exchanged between agents



Best/worst bid exchanged



Utility space

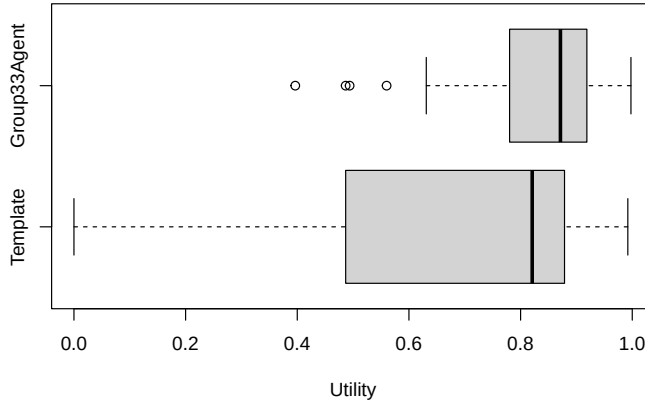


The final plot above, showing the negotiation domain, shows our agent’s successful convergence to a consensus on the Pareto front.

4.3 Comparison to the baseline agent

The following plot compares all runs against opponent agents for the baseline implementation (the ANAC 2023 *TemplateAgent*) and the agent described in this report (for *Group33Agent*). The sample size is all 50 opponents for *domain26*.

Comparison to the baseline implementation



5 Limitations

5.1 Limitations of the evaluation

Our evaluation used over 5000 runs to generate evaluation data that compared our agent to other agents in the repository. We have identified the limitations of this procedure as well as questions left to address in future work.

Incomplete negotiations

During the agent’s training, we decided to manually set the acceptance utility threshold (reservation value) to 0.6. In future, we would consider changing this to a dynamic value because it does not suit all domains. In particular, some negotiations rejected in no offer being accepted as our agent would refuse to concede (most notably to *HardlineAgent*).

Learning agents

The data suggests good performance in one-shot learning agents. However, the evaluation does not consider advanced learning agents that could presumably gain an advantage over a number of ‘training’ runs against our implementation.

5.2 Limitations of the agent’s implementation

Long-term strategy detection

One explanation for why our agent did not perform well against the most advanced agents on offer, is due to the fact that we are only considering the last five bids in determining opponent behaviour for bidding purposes. While this works in our favor in fighting against mixed opponent strategies, this can prevent us from adapting to opponents who strategically alternate between long periods of tough and conceding tactics.

Support for other utility spaces

Our agent currently supports only linear additive utility spaces, so non-linear or interdependent issues are not considered.

Utility space sample size

In addition to the above, we are only sampling 1000 random bids, which may become problematic in large domains with many issues. This approach could fail to explore the offer space accurately. We suggest that this parameter be adjusted appropriately to the sample space size.

Acceptance Strategy

Our acceptance strategy might accept bids too early. When there is only 10% of the time remaining, we accept any offer that is better than all previous ones. This might miss opponents that started off unwilling to negotiate, but end up conceding more towards the end, causing us to accept a non optimal offer.

6 Conclusion

In conclusion, we have shown that a DANS-based opponent model is a viable strategy for autonomous negotiation. We proposed an agent based on the BOA architecture, incorporating DANS as well as novel strategies for the opponent model and bidding strategy. We performed Bayesian optimization and selected parameters for the competition. Our agent was evaluated against 50 ANAC 2023 agents over 50 domains and showed above-average performance in unseen domains, cementing the agent’s architecture as a valid new approach to bilateral negotiation.

Ethical Statement

While there are no ethical issues with the design and implementation of our autonomous agent, Baarslag et al. (2017) writes that such issues may arise when self-serving negotiation agents such as ours are applied in sensitive contexts.

Acknowledgments

We acknowledge the support and instruction of Carolina Jorge, Ruben Verhagen, Myrthe Tielman, and Pradeep K. Murukanaiah of Interactive Intelligence at Delft University of Technology. We thank the Teaching Assistants.

Work distribution

Group 33 distributed the work equally.

References

- Aydogan, Reyhan, David Festen, Koen V. Hindriks, and Catholijn M. Jonker. 2017. “Alternating Offers Protocols for Multilateral Negotiation.” In *Modern Approaches to Agent-Based Complex Automated Negotiation*, edited by Katsuhide Fujita, Quan Bai, Takayuki Ito, Minjie Zhang, Fenghui Ren, Reyhan Aydoğan, and Rafik Hadfi, 1st ed., 153–67. Studies in Computational Intelligence. Springer. https://doi.org/10.1007/978-3-319-51563-2_10.
- Baarslag, Tim. 2016. *Exploring the Strategy Space of Negotiating Agents*. Springer Theses. Springer International Publishing. <https://doi.org/10.1007/978-3-319-28243-5>.
- Baarslag, Tim, Koen Hindriks, and Catholijn Jonker. 2014. “Effective Acceptance Conditions in Real-Time Automated Negotiation.” *Decision Support Systems* 60 (April): 68–77. <https://doi.org/10.1016/j.dss.2013.05.021>.
- Baarslag, Tim, Michael Kaisers, Enrico Gerding, Catholijn M. Jonker, and Jonathan Gratch. 2017. “When Will Negotiation Agents Be Able to Represent Us? The Challenges and Opportunities for Autonomous Negotiators.” In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, edited by Carles Sierra, 4684–90.

International Joint Conferences on Artificial Intelligence.
<https://eprints.soton.ac.uk/412727/>.

- [] Faratin, Peyman, Carles Sierra, and Nick R. Jennings. 1998. “Negotiation Decision Functions for Autonomous Agents.” *Robotics and Autonomous Systems* 24 (3–4): 159–82. [http://doi.org/10.1016/s0921-8890\(98\)00029-3](http://doi.org/10.1016/s0921-8890(98)00029-3).
- [] Li, Ming, Pradeep K. Murukannaiah, and Catholijn M. Jonker. 2021. “A Data-Driven Method for Recognizing Automated Negotiation Strategies.” arXiv. <https://doi.org/10.48550/ARXIV.2107.01496>.

Appendix A: Optimization Parameters

The following parameters were optimized using Bayesian Optimization:

- | | |
|----------------------|----------------------------|
| • t | • default_eps |
| • t_prime | • FORTUNATE_params_alpha |
| • final_utility | • SELFISH_params_alpha |
| • U_max | • CONCESSION_params_alpha |
| • U_min | • UNFORTUNATE_params_alpha |
| • b_early | • NICE_params_alpha |
| • b_late | • SILENT_params_alpha |
| • FORTUNATE_offset | • FORTUNATE_params_eps |
| • SELFISH_offset | • SELFISH_params_eps |
| • CONCESSION_offset | • CONCESSION_params_eps |
| • UNFORTUNATE_offset | • UNFORTUNATE_params_eps |
| • NICE_offset | • NICE_params_eps |
| • SILENT_offset | • SILENT_params_eps |
| • default_alpha | |

The agent was evaluated against the following opponents:

- | | |
|------------------|------------|
| • BoulwareAgent | • Agent26 |
| • ConcederAgent | • Agent27 |
| • HardlinerAgent | • Agent29 |
| • LinearAgent | • Agent32 |
| • RandomAgent | • Agent33 |
| • StupidAgent | • Agent41 |
| • Agent2 | • Agent43 |
| • Agent3 | • Agent50 |
| • Agent7 | • Agent52 |
| • Agent11 | • Agent55 |
| • Agent14 | • Agent58 |
| • Agent18 | • Agent61 |
| • Agent19 | • Agent64 |
| • Agent22 | • Agent67 |
| • Agent24 | • Agent68 |
| • Agent25 | • Agent007 |

- Agent4410
- AgentFish
- AgentFO2
- BIU_agent
- ChargingBoul
- CompromisingAgent
- DreamTeam109Agent
- GEAAgent
- LearningAgent
- LuckyAgent2022
- MiCROAgent
- Pinar_Agent
- ProcrastinAgent
- RGAgent
- SmartAgent
- SuperAgent
- ThirdAgent
- Tjaronchery10Agent